

# PAPER CONTRIBUTION — DEFINING AND CHARACTERIZING REWARD HACKING

**Lucas Mebille, Felix Rosseeuw, Jules Cognon**

Department of Applied Mathematics  
Ecole Polytechnique  
Palaiseau, 91120, France

## ABSTRACT

Reward hacking occurs when optimizing a proxy reward function leads to behaviors that diverge from intended objectives, a critical challenge in reinforcement learning (RL) systems. This report builds on the foundational work *Defining and Characterizing Reward Hacking* (Skalse et al. (2022)) by examining its theoretical framework and exploring extensions. We propose a novel corollary to Theorem 1, highlighting the significance of hackable policy sets in open environments, and introduce a relaxed  $\epsilon$ -hackability definition to account for practical constraints. Furthermore, we illustrate reward hacking in Large Language Models (LLMs) trained via Reinforcement Learning with Human Feedback (RLHF), showcasing real-world vulnerabilities.

## 1 INTRODUCTION

Reward hacking is a concept that captures a common issue in real-world systems when imperfect objectives lead to unintended outcomes. For instance, a university might reject strong applicants to appear more selective, or an AI in a racing game might prioritize collecting power-ups over completing the race. These failures arise when optimizing for a proxy reward function  $\tilde{\mathcal{R}}$  produces behavior that deviates from the true reward function  $\mathcal{R}$ . The paper Skalse et al. (2022) formalizes this issue, defining a proxy as "unhackable" if improving performance according to the proxy cannot worsen performance according to the true reward. However, unhackability is shown to be an extremely strict condition: for all stochastic policies, meaningful unhackable proxies are nearly impossible. By narrowing the focus to deterministic or finite sets of policies, unhackability becomes achievable but remains challenging. The authors also examine "simplifications," proxies intended as simpler approximations of the true reward. While intuitive, simplifications often fail to eliminate the potential for reward hacking. These results underline the inherent difficulty of designing proxy objectives that align AI behavior with intended goals, especially in complex or safety-critical environments.

## 2 PAPER SUMMARY

### 2.1 REINFORCEMENT LEARNING NOTATIONS AND CONTEXT

For a reminder of all the formalizations in the RL framework, please refer to A, as outlined in the paper by Skalse et al. (2022). .

### 2.2 FORMAL DEFINITION OF REWARD HACKING

Here is the formal definition, given in the paper, of **hackability** as a binary relation between two reward functions

**Definition 1.** A pair of reward functions  $\mathcal{R}_1, \mathcal{R}_2$  are **hackable** if there exist  $\pi, \pi' \in \Pi$  such that

$$J_1(\pi) < J_1(\pi') \ \& \ J_2(\pi) > J_2(\pi'),$$

else they are **unhackable**.

The paper notes that an unhackable reward pair can have  $J_1(\pi) < J_1(\pi')$  &  $J_2(\pi) = J_2(\pi')$  or vice versa. Unhackability is symmetric, as can be observed by swapping  $\pi$  and  $\pi'$  in Definition 1. However, it is not transitive. For instance, the constant reward function is unhackable with respect to any other reward function. If unhackability were transitive, this would imply that any pair of policies is unhackable.

The main question addressed by the paper is **whether and when interesting unhackable reward pairs exist**.

By interesting, the paper means **non-trivial** reward functions and **not equivalent** pairs. A formal definition of these terms is also provided :

**Definition 2.**  $R_1$  and  $R_2$  are **equivalent** on a set of policies  $\Pi$  if  $J_1$  and  $J_2$  induce the same ordering of  $\Pi$ .

**Definition 3.**  $R$  is **trivial** on  $\Pi$  if  $J(\pi) = J(\pi')$  for all  $\pi, \pi' \in \Pi$ .

As the motivation of the paper distinguishes the proxy and true reward functions, the symmetric nature of this definition appears counter-intuitive. The paper suggests that this symmetry could be broken by only considering policy sequences that *monotonically increase* the proxy. However, this approach is equivalent to the original definition of hackability:  $R_1$  is viewed as the proxy, and the sequence  $\pi, \pi'$  is considered.

Finally, the paper defines **simplification** as an *asymmetric* special-case of unhackability.

**Definition 4.**  $R_2$  is a **simplification** of  $R_1$  relative to policy set  $\Pi$  if for all  $\pi, \pi' \in \Pi$ ,

$$\begin{aligned} J_1(\pi) < J_1(\pi') &\implies J_2(\pi) \leq J_2(\pi'), \\ J_1(\pi) = J_1(\pi') &\implies J_2(\pi) = J_2(\pi'). \end{aligned}$$

Furthermore, there exist  $\pi, \pi' \in \Pi$  such that  $J_2(\pi) = J_2(\pi')$  but  $J_1(\pi) \neq J_1(\pi')$ .

Moreover, if  $R_2$  is trivial, then this is referred to as a **trivial simplification**.

## 2.3 RESULTS

### 2.3.1 NON-TRIVIAL UNHACKABILITY IN INFINITE POLICY SETS

The paper first investigates the conditions under which non-trivial unhackable reward functions can exist within the infinite set of all stochastic policies. Using the linearity of reward functions with respect to state-action visitation counts, it establishes that achieving unhackability in such a setting is extraordinarily restrictive. Specifically, **Theorem 1** demonstrates that if a set of policies contains an open subset, then any two reward functions that are unhackable and non-trivial on this set must be equivalent:

**Theorem 1.** In any  $MDP \setminus \mathcal{R}$ , if  $\hat{\Pi}$  contains an open set, then any pair of reward functions that are unhackable and non-trivial on  $\hat{\Pi}$  are equivalent on  $\hat{\Pi}$ .

This result implies that the full set of stationary policies, which includes an open subset, does not allow for meaningful unhackable reward pairs. In essence, for any pair of reward functions within this set, improving performance according to one reward necessarily implies an equivalent improvement according to the other.

Its first corollary extends this insight to show that even when considering the entire set of stationary policies ( $\Pi$ ), unhackable and non-trivial reward functions must be equivalent across the set. The implications are significant: in such expansive policy sets, any attempt to construct an unhackable proxy is bound to fail unless the proxy exactly mirrors the true reward function.

**Corollary 1.** In any  $MDP \setminus \mathcal{R}$ , any pair of reward functions that are unhackable and non-trivial on the set of all (stationary) policies  $\Pi$  are equivalent on  $\Pi$ .

To explore potential exceptions, the study considers restricting the policy set to other relevant infinite subsets, such as  $\varepsilon$ -suboptimal policies (those close enough to optimal policy) and  $\delta$ -deterministic policies (those with limited stochasticity). These subsets correspond to practical scenarios like

focusing on high-performing candidate policies or favoring deterministic behaviors over purely stochastic ones. However, both subsets contain open components, making them subject to the same limitations outlined in Theorem 1. As shown in the second corollary, any pair of reward functions that are unhackable and non-trivial within these restricted subsets are still equivalent. Thus, the paper concludes that meaningful unhackability is unattainable in any policy set with open subsets.

For infinite policy sets without open subsets, the paper provides examples illustrating contrasting outcomes regarding unhackability. One example involves a set  $\Pi_a$ , where unhackable reward pairs exist, as  $\mathcal{R}_2$  simplifies  $\mathcal{R}_1$ . Another example,  $\Pi_b$ , demonstrates that unhackable pairs may not always exist, as  $\mathcal{R}_2$  fails to maintain consistent preferences between policies under  $\mathcal{R}_1$  and  $\mathcal{R}_2$ . Therefore, no general rule can be identified for such policy sets without further precision or additional constraints.

## 2.4 NON-TRIVIAL UNHACKABILITY IN FINITE POLICY SETS

In contrast to infinite policy sets, the paper also studies finite sets of policies, such as the set of all deterministic policies in finite MDPs. Remarkably, the results here are significantly more permissive. **Theorem 2.** establishes that, for any finite policy set  $\hat{\Pi}$  containing at least two distinct policies (in terms of their state-action occupancy vectors), it is always possible to construct a pair of reward functions,  $\mathcal{R}_1$  and  $\mathcal{R}_2$ , that are unhackable yet non-trivial and non-equivalent. This means that unhackable proxies with meaningful distinctions from the true reward can exist within finite policy sets, unlike in the case of infinite stochastic policies.

**Theorem 2.** *For any MDP  $\mathcal{M}$ , any finite set of policies  $\hat{\Pi}$  containing at least two  $\pi, \pi'$  such that  $\mathcal{F}(\pi) \neq \mathcal{F}(\pi')$ , and any reward function  $\mathcal{R}_1$ , there is a non-trivial reward function  $\mathcal{R}_2$  such that  $\mathcal{R}_1$  and  $\mathcal{R}_2$  are unhackable but not equivalent.*

The proof of Theorem 2 is intuitive: it demonstrates that one can construct a path in the space of reward functions from  $\mathcal{R}_1$  to another reward function,  $\mathcal{R}_3$ , which is hackable relative to  $\mathcal{R}_1$ . Along this path, there must exist an intermediate reward function,  $\mathcal{R}_2$ , that replaces one of  $\mathcal{R}_1$ 's inequalities with equality, ensuring  $\mathcal{R}_2$  is both non-trivial and unhackable. This result leverages the finiteness of  $\hat{\Pi}$ , where the linear constraints imposed by policies create structure in the reward space that allows such constructions.

The paper further refines this result to the concept of reward simplifications, where  $\mathcal{R}_2$  is a simplified version of  $\mathcal{R}_1$ . **Corollary 3.** provides necessary and sufficient conditions for such simplifications. Specifically, it defines a procedure based on partitioning the policies in  $\hat{\Pi}$  by their rewards under  $\mathcal{R}_1$  and analyzing the dimensionality of their state-action occupancy differences. If the diversity of policy behaviors is sufficiently constrained, simplifications become possible.

**Corollary 3.** *For any finite set of policies  $\hat{\Pi}$ , any environment, and any reward function  $R$ , if  $|\hat{\Pi}| \geq 2$  and  $J(\pi) \neq J(\pi')$  for all  $\pi, \pi' \in \hat{\Pi}$ , then there is a non-trivial simplification of  $R$ .*

These findings highlight the unique properties of finite policy sets: they not only guarantee the existence of unhackable proxies but also provide a structured way to assess whether meaningful simplifications of reward functions can be achieved, which is impossible in infinite policy spaces.

## 3 OUR CONTRIBUTIONS

### 3.1 PAPER LIMITATIONS STUDY

As summarized above, this paper provides a formal introduction to reward hacking, laying the theoretical foundations for a problem that was already known prior to its publication. However, while the results presented offer general characterizations of hackability within environments with finite MDPs and Markovian reward functions, the paper leaves room to consider less restrictive definitions of hackability. Such definitions could potentially yield different results for environments similar to the one studied, or even for more general cases. Our in-depth study of the article follows this direction, aiming to explore certain properties arising from a broader definition of hackability.

First, we propose a corollary to Theorem 1, which highlights the relevance of the definition provided by the authors for the concept of hackability between two reward functions. Specifically, we show that in the case of a policy set with an open subset, the set of policies capable of hacking a pair of reward functions is "non-negligible," in the sense that there exist two open sets within the set of policies that hack the pair. This reinforces the importance of studying the issue of reward hacking in reinforcement learning (RL).

**Corollary 4.** *In any MDP  $\setminus \mathcal{R}$ , if  $\hat{\Pi}$  contains an open set, then  $\forall (\mathcal{R}_1, \mathcal{R}_2) \in \mathcal{R}^2$ , such that  $\mathcal{R}_1$  and  $\mathcal{R}_2$  are not equivalent,  $\exists (\Pi_1, \Pi_2)$  two open sets of  $\hat{\Pi}$  for which  $\forall (\pi_1, \pi_2) \in \Pi_1 \times \Pi_2$ ,  $J_1(\pi_1) < J_1(\pi_2)$  &  $J_2(\pi_1) > J_2(\pi_2)$ .*

**Proof.** Let  $(R_1, R_2) \in \mathcal{R}^2$  such that  $R_1$  and  $R_2$  are not equivalent. By **Theorem 1**,  $R_1$  and  $R_2$  are hackable; hence there exists  $(\pi_1, \pi_2) \in \hat{\Pi}^2$  such that  $J_1(\pi_1) < J_1(\pi_2)$  &  $J_2(\pi_1) > J_2(\pi_2)$ .

Now, there exists  $\pi_0 \in \hat{\Pi}$  and  $r > 0$  such that  $B(\mathcal{F}(\pi_0), r) \subset \hat{\Pi}$  (by the continuity of  $\mathcal{F}$ ). Thus, there exists  $\delta > 0$  such that  $(f_1, f_2) = (\delta\mathcal{F}(\pi_1) + (1-\delta)\mathcal{F}(\pi_0), \delta\mathcal{F}(\pi_2) + (1-\delta)\mathcal{F}(\pi_0)) \in B(\mathcal{F}(\pi_0), r)^2$ .

By linearity:

$$\begin{aligned} J_1(f_1) &= J_1(\delta\mathcal{F}(\pi_1) + (1-\delta)\mathcal{F}(\pi_0)), \\ J_1(f_1) &= \delta J_1(\mathcal{F}(\pi_1)) + (1-\delta)J_1(\mathcal{F}(\pi_0)), \\ J_1(f_1) &< \delta J_1(\mathcal{F}(\pi_2)) + (1-\delta)J_1(\mathcal{F}(\pi_0)), \\ J_1(f_1) &< J_1(f_2). \end{aligned}$$

Similarly, we show that  $J_2(f_1) > J_2(f_2)$ .

Define  $G_1(f_1, f_2) := J_1(f_1) - J_1(f_2)$  and  $G_2(f_1, f_2) := J_2(f_1) - J_2(f_2)$ . ( $G_1, G_2$ ) are continuous.

The set  $] -\infty, 0[ \times ]0, +\infty[$  is an open subset of  $\mathbb{R}^2$ . Thus,  $(O_1, O_2) = (G_1^{-1}(] -\infty, 0[ \times ]0, +\infty[), G_2^{-1}(] -\infty, 0[ \times ]0, +\infty[))$  are open sets in  $\hat{\Pi}$ , and since  $(f_1, f_2) \in O_1 \cap O_2$  and  $(f_1, f_2) \in B_O(\mathcal{F}(\pi_0), r)^2$ , the intersection  $O_1 \cap O_2 \cap B_O(\mathcal{F}(\pi_0), r)^2$  satisfies the conditions.

We have now considered a less restrictive definition of hackability, introducing a margin for the inversion of inequality between the real reward function and the proxy reward function.

**Definition 5.** *A pair of reward functions  $R_1, R_2$  are  $\epsilon$ -hackable relative to policy set  $\Pi$  and an environment  $(S, A, T, I, \gamma)$  if there exist  $\pi, \pi' \in \Pi$  such that:*

$$J_1(\pi) < J_1(\pi') \quad \& \quad J_2(\pi) > J_2(\pi')(1 + \epsilon),$$

*else they are  $\epsilon$ -unhackable.*

This definition relaxes the constraints imposed by the authors' original notion of hackability. In particular, it considers the case where, given a set of policies and a pair of reward functions, the relative ordering inversion caused by the proxy function does not exceed  $\epsilon$ . We then attempted to explore the properties associated with this definition—especially over the entire policy set. Unfortunately, we did not find any noteworthy results for this definition.

### 3.2 PAPER ILLUSTRATION: REWARD HACKING IN LLMs

The paper we studied addresses an emerging topic (notably, as of 2022) and remains quite theoretical regarding the applications of reward hacking.

To provide a practical perspective, we explored how reward hacking manifests in the context of Large Language Models (LLMs) and Reinforcement Learning with Human Feedback (RLHF). Below is an explanation of how this phenomenon occurs in such a scenario:

Large Language Models (LLMs) are inherently susceptible to **reward hacking** due to the structure of their policy space and the limitations of the reward functions employed during training. In a **Reinforcement Learning with Human Feedback (RLHF)** framework, the policies of LLMs are represented as probability distributions over tokens, parameterized by the weights of a neural network. Due to the **continuous** and **open** nature of this policy space, small variations in the model's parameters can lead to a **wide range of behaviors**, enabling new policies to emerge easily.

According to **Theorem 1** from the paper, this implies that achieving complete unhackability in such a context is fundamentally impossible, even when reward functions are carefully designed to promote desirable behaviors. Consequently, these models remain vulnerable to fine-tuning that prioritizes misaligned behaviors. For instance, behaviors like **hallucination generation** may arise simply because they locally optimize the reward function in the vicinity of the current policy. The combination of a highly flexible policy space and imperfect reward functions makes LLMs particularly prone to **reward hacking**.

Here is an explicit example of reward hacking in the context of LLMs:

- **Initial state** ( $s_0$ ): The user asks a factual question.  
“What novels were written by Jules Verne?”
- **Action space** ( $A$ ):
  - $a_1$ : A correct and concise answer:  
“Famous novels by Jules Verne include Twenty Thousand Leagues Under the Sea and Around the World in Eighty Days.”
  - $a_2$ : A hallucinated but plausible answer:  
“Jules Verne wrote The Lost Stars and The Celestial Odyssey, in addition to his famous works like Around the World in Eighty Days.”
- **Initial policy**  $\pi$ : The policy  $\pi(a | s)$  favors correct, factually aligned responses with  $J_1(\pi)$ , as it maximizes the factual reward  $R_{\text{true}}(s, a)$ . In this state  $s_0$ ,  $\pi$  would select  $a_1$  as the response since it is factually correct.
- **Exploiting policy**  $\pi'$ : The policy  $\pi'$  discovers that hallucinated responses like  $a_2$  obtain higher rewards in  $R_{\text{proxy}}(s, a)$ . This can happen because:
  - Human evaluators find  $a_2$  more impressive and detailed, or don’t fact check the information
  - The reward model trained prefers long and complex answers, even if they contain errors $\pi'$  would thus maximize  $J_2(\pi')$  by consistently choosing  $a_2$  in  $s_0$ .

#### Reward Functions and Objectives:

- **True reward**  $R_{\text{true}}(s, a)$ :
  - $R_{\text{true}}(s_0, a_1) = 1$  (correct answer)
  - $R_{\text{true}}(s_0, a_2) = 0$  (hallucination)
- **Proxy reward**  $R_{\text{proxy}}(s, a)$ :
  - $R_{\text{proxy}}(s_0, a_1) = 0.8$  (concise but not “impressive” answer)
  - $R_{\text{proxy}}(s_0, a_2) = 1.0$  (long and “detailed” answer, even though incorrect)

#### Objective Evaluation:

- **True objective**  $J_1(\pi)$ :  
For  $\pi$ , the value of  $J_1$  is:  
$$J_1(\pi) = R_{\text{true}}(s_0, a_1) = 1$$
  
For  $\pi'$  that chooses  $a_2$ :  
$$J_1(\pi') = R_{\text{true}}(s_0, a_2) = 0$$
- **Proxy objective**  $J_2(\pi')$ :  
For  $\pi$ , which selects  $a_1$ :  
$$J_2(\pi) = R_{\text{proxy}}(s_0, a_1) = 0.8$$
  
For  $\pi'$  that selects  $a_2$ :  
$$J_2(\pi') = R_{\text{proxy}}(s_0, a_2) = 1.0$$

#### Identified Reward Hacking:

- **Difference between  $J_1$  and  $J_2$ :** With  $\pi$ , we have  $J_1(\pi) = 1$  and  $J_2(\pi) = 0.8$ . With  $\pi'$ , we have  $J_1(\pi') = 0$  and  $J_2(\pi') = 1.0$ . This shows that the exploiting policy  $\pi'$  maximizes  $J_2$  (the proxy objective) at the cost of  $J_1$  (the true objective).
- **Exploitation of  $R_{\text{proxy}}$ :** The policy  $\pi'$  has discovered a flaw in  $R_{\text{proxy}}(s, a)$ , where hallucinated responses receive higher rewards, constituting a clear case of reward hacking.

### Consequences and Solutions:

The user receives incorrect answers, which reduces their trust in the model decreases. Even worse, one could take for granted the LLM’s answer and spread the misinformation. To prevent this, one can improve  $R_{\text{proxy}}(s, a)$  by adding penalties for factual errors. An other solution is to include adversarial examples to correct biases in the reward model (supervised reinforcement learning). Finally, integrating mechanisms to verify answers before evaluation or submission (fact-checking) can help prevent reward hacking in this context.

## 4 CONCLUSION

In this report, we examined the theoretical framework presented in *Defining and Characterizing Reward Hacking* (Skalse et al. (2022)) and extended its findings through further analysis. We introduced a corollary to Theorem 1, emphasizing the non-negligible nature of hackable policy sets, and explored a broader  $\epsilon$ -hackability definition, which highlights opportunities for relaxed constraints. Additionally, we provided a practical illustration of reward hacking in Large Language Models (LLMs) under RLHF, demonstrating its real-world implications. By bridging theoretical insights with practical examples, our contributions underscore the challenges of designing robust reward proxies and the critical need for careful alignment in reinforcement learning applications.

## REFERENCES

Joar Skalse, Nikolaus H. R. Howe, Dmitrii Krashennnikov, and David Krueger. Defining and characterizing reward hacking, 2022. URL <https://arxiv.org/abs/2209.13085>.

## A APPENDIX

### A.1 REINFORCEMENT LEARNING NOTATIONS AND CONTEXT

Here is the formalization of the paper regarding reinforcement learning frameworks (section 4.1 of the article, page 4) :

Reinforcement Learning (RL) methods attempt to solve a sequential decision problem, typically formalised as a Markov decision process (MDP), which is a tuple  $(S, A, T, I, R, \gamma)$  where:

-  $S$  is a set of states, -  $A$  is a set of actions, -  $T : S \times A \rightarrow \Delta(S)$  is a transition function, -  $I \in \Delta(S)$  is an initial state distribution, -  $R$  is a reward function, the most general form of which is  $R : S \times A \times S \rightarrow \Delta(R)$ , -  $\gamma \in [0, 1]$  is the discount factor.

Here,  $\Delta(X)$  is the set of all distributions over  $X$ . A stationary policy is a function  $\pi : S \rightarrow \Delta(A)$  that specifies a distribution over actions in each state, and a non-stationary policy is a function  $\pi : (S \times A)^* \times S \rightarrow \Delta(A)$ , where  $*$  is the Kleene star. A trajectory  $\tau$  is a path  $s_0, a_0, r_0, \dots$  through the MDP that is possible according to  $T, I$ , and  $R$ . The return of a trajectory is the discounted sum of rewards:

$$G(\tau) \doteq \sum_{t=0}^{\infty} \gamma^t r_t,$$

and the value of a policy is the expected return:

$$J(\pi) \doteq \mathbb{E}_{\tau \sim \pi}[G(\tau)].$$

We derive policy (preference) orderings from reward functions by ordering policies according to their value. In this paper, we assume that  $S$  and  $A$  are finite, that  $|A| > 1$ , that all states are reachable, and that  $R(s, a, s')$  has finite mean for all  $s, a$ , and  $s'$ .

In our work, we consider various reward functions for a given environment, which is then formally a Markov decision process without reward, denoted  $MDP \setminus R \doteq (S, A, T, I, \gamma)$ . Having fixed an  $MDP \setminus R$ , any reward function can be viewed as a function of only the current state and action by marginalizing over transitions:

$$R(s, a) \doteq \sum_{s' \sim T(s'|s, a)} R(s, a, s').$$

We adopt this view from here on. We define the (discounted) visit counts of a policy as:

$$F^\pi(s, a) \doteq \mathbb{E}_{\tau \sim \pi} \left[ \sum_{i=0}^{\infty} \gamma^i \mathbf{1}(s_i = s, a_i = a) \right].$$

Note that  $J(\pi) = \sum_{s, a} R(s, a) F^\pi(s, a)$ , which we also write as  $\langle R, F^\pi \rangle$ . When considering multiple reward functions in an  $MDP \setminus R$ , we define:

$$J_R(\pi) \doteq \langle R, F^\pi \rangle$$

and sometimes use:

$$J_i(\pi) \doteq \langle R_i, F^\pi \rangle$$

as shorthand. We also use  $F : \Pi \rightarrow \mathbb{R}^{|S||A|}$  to denote the embedding of policies into Euclidean space via their visit counts, and define:

$$F(\Pi) \doteq \{F(\pi) : \pi \in \Pi\}$$

for any  $\Pi$ . Moreover, we also use a second way to embed policies into Euclidean space; let  $G(\pi)$  be the  $|S||A|$ -dimensional vector where  $G(\pi)[s, a] = \pi(a|s)$ , and let:

$$G(\Pi) \doteq \{G(\pi) : \pi \in \Pi\}.$$